```
        ret_ = torch.Tensor([0])
        for r in range(rewards.shape[0]):
2
            ret_ = rewards[r] + gamma * ret_
            Returns.append(ret_)
        Returns = torch.stack(Returns).view(-1)
        Returns = F.normalize(Returns,dim=0)
        actor_loss = -1*logprobs * (Returns - values.detach())
3
        critic_loss = torch.pow(values - Returns,2)
4
        loss = actor_loss.sum() + clc*critic_loss.sum()
5
        loss.backward()
        worker_opt.step()
        return actor_loss, critic_loss, len(rewards)
```

- ***1*** We reverse the order of the rewards, logprobs, and values_ arrays and call .view(-1) to make sure they're flat.
- ***2*** For each reward (in reverse order), we compute the return value and append it to a returns array.
- ***3*** We need to detach the values tensor from the graph to prevent backpropagating through the critic head.
- ***4*** The critic attempts to learn to predict the return.
- ***5*** We sum the actor and critic losses to get an overall loss. We scale down the critic loss by the clc factor.

The `update_params` function is where all the action is, and it's what sets distributed advantage actor-critic apart from the other algorithms we've learned so far. First we take the lists of rewards, log probabilities, and state values and convert them to PyTorch tensors. We then reverse their order because we want to consider the most recent action first, and we make sure they are flattened 1D arrays by calling the `.view(-1)` method.

The `actor_loss` is computed as we described earlier in this section with math, using the advantage (technically the baseline, since there's no bootstrapping) rather than the raw reward. Crucially, we must detach the values tensor from the graph when we use the `actor_loss`, or we will backpropagate through the actor and critic heads, and we only want to update the actor head. The critic loss is a simple squared error between the state values and the returns, and we make sure *not* to detach here since we